

CircleUp Implementation Roadmap — Solo Entrepreneur Edition

Context

CircleUp is fully designed in Design OS: 17 sections specified, 169+ screen design components, 22+ data model entities, complete shell and design system. This roadmap sequences the **actual implementation** for a solo developer using **Ruby on Rails + React + Tailwind CSS**, targeting a 6-month MVP with real diaspora associations as early adopters.

The core insight: CircleUp's value is the **contribution -> payout loop**. Everything else supports that. The roadmap prioritizes accordingly.

Tech Stack

Layer	Choice	Why
Backend	Ruby on Rails 7 (API mode)	Convention over config, rapid scaffolding, mature ecosystem
Frontend	React + TypeScript + Vite	169 Design OS components as reference implementations
Styling	Tailwind CSS v4	Already used in Design OS, indigo/amber/slate tokens
Database	PostgreSQL	UUIDv7 PKs, JSONB settings, decimal money fields
Jobs	Sidekiq + Redis	Payment reminders, Trust Score recalculation, payouts
Payments	Stripe Connect	Contribution collection + payout disbursement
Auth	Devise + devise-jwt	JWT in httpOnly cookies, Google OAuth
Authorization	Pundit	Policy-based RBAC
Audit	paper_trail	Automatic version tracking (replaces manual AuditLog)
Hosting	Railway or Render	~\$25-50/month, managed Postgres + Redis, auto-deploy
Email	SendGrid	Free tier (100/day), transactional emails
Errors	Sentry	Free tier, Slack alerting
CDN/DNS	Cloudflare	Free tier, SSL, DDoS protection

Total infrastructure cost: ~\$35-60/month

Phase 0: Foundation (Weeks 1-3)

Week 1: Project Scaffolding & CI/CD

Backend:

- `rails new circleup --api --database=postgresql`
- UUIDv7 default primary keys
- Core gems: `devise-jwt`, `pundit`, `stripe`, `sidekiq`, `pagy`, `ransack`, `paper_trail`, `discard` (soft deletes), `aasm` (state machines), `rspec-rails`, `factory_bot_rails`
- `rack-cors` for React frontend

Frontend:

- Vite + React + TypeScript
- Tailwind CSS v4 (no config file per project conventions)
- React Router v7, Tanstack Query, Zustand
- Lucide React icons, Inter + JetBrains Mono fonts
- Shared component library: Button, Input, Card, Badge, Avatar, Modal, Toast, DataTable, EmptyState

Infrastructure:

- GitHub repo with branch protection
- GitHub Actions CI (RSpec + ESLint + tsc)
- Staging environment with auto-deploy on merge
- Sentry error tracking

Done when: App running at staging URL, CI green, auto-deploy working.

Week 2: Core Database Schema (12 MVP entities)

Create Rails models in dependency order:

1. **User** — email, password, name, phone, avatar, trust_score, kyc_status, stripe_customer_id, settings:jsonb
2. **Association** — name, description, logo, type, visibility, country, language, settings:jsonb
3. **Membership** — user, association, role, status, membership_type, dues_paid_until
4. **Circle** — association, name, contribution_amount, frequency, duration, max_participants, current_cycle, status, payout_method, emergency_fund_rate, balances, settings:jsonb
5. **Participant** — user, circle, role, payout_position, payment_counts, total_contributed, payout_received
6. **Contribution** — participant, circle, cycle_number, amount, emergency_fund_portion, due_date, status, stripe_payment_intent_id
7. **Payout** — participant, circle, cycle_number, amounts, status, scheduled_date, stripe_transfer_id
8. **EmergencyFundIntervention** — circle, defaulting_participant, covered_contribution, amounts, status
9. **PaymentMethod** — user, type, last_four, brand, is_default, stripe_payment_method_id
10. **Transaction** — association, circle, user, type, direction, amount, currency, status, metadata:jsonb
11. **Notification** — user, type, title, body, channel, read
12. **AuditLog** — handled automatically by paper_trail

Deferred entities: WelfareFund, WelfareApplication, Event, Election, Proposal, Vote, Message, Document, Federation

Done when: All models with validations, associations, factories, passing specs. `rails db:seed` creates realistic sample data.

Week 3: Authentication & App Shell

Auth (Rails):

- Registration with email verification (letter_opener dev, SendGrid prod)
- Login returning JWT in httpOnly cookie
- Password reset flow
- Google OAuth via omniauth
- Account lockout after 5 failed attempts

Auth (React) — from Login section's 10 components:

- AuthLayout, LoginForm, RegisterForm, ForgotPasswordForm, ResetPasswordForm
- SocialButtons (Google only), PasswordInput, VerificationPending
- OnboardingWizard — simplify to 3 steps: photo, phone, language

App Shell — from `src/shell/` components:

- Collapsible sidebar (252px / 60px)
- 2 MVP roles: `member` and `organizer` (defer granular RBAC)
- Nav groups: Home, Core (Associations, Members, Circles), Finance (Treasury)
- User menu with avatar and role
- Mobile: off-canvas drawer

Done when: User can register, verify email, log in, see shell with navigation, complete onboarding.

Phase 1: MVP Core (Months 1-3, Weeks 4-12)

MVP includes 6 sections — the minimum for the ROSCA value loop:

Section	Design OS Components	MVP Components	Effort
Login	10	10	Week 3 (done)
Associations	20	12	Weeks 4-5
Members & Trust	8	6	Weeks 6-7
ROSCA Circles	16	10	Weeks 8-10
Treasury & Funds	8	4	Week 11
Notifications	(from Comm)	3	Week 12

Weeks 4-5: Associations

Rails: AssociationsController (CRUD), MembershipsController (join/leave/roles), AssociationPolicy, invitation system with unique codes/links

React (12 of 20 components):

- MyAssociationsList, AssociationCard, AssociationDashboard
- CreateAssociation — "from scratch" only (defer migration wizard)
- AssociationSettings (name, description, type, visibility)
- MemberDirectory, InviteMembers, RoleBadge, AssociationTypeBadge
- ActivityFeed, DiscoverAssociations (stub)

Defer: MigrationDashboard, MigrationWizard, AnnouncementCard, AssociationAnalytics, AssociationAdministration, FederationDashboard, ChapterDirectory, PolicyManager

Weeks 6-7: Members & Trust

Rails: MembersController, TrustScoresController, `TrustScoreCalculator` service (MVP: weighted formula, not ML)

Trust Score formula (MVP):

- Payment history (40%): `on_time / total_contributions`

- Verification (20%): profile completeness + email/phone verified
- Tenure (15%): months since joining
- Engagement (10%): login frequency
- Network (10%): circles participated in
- External (5%): stub at 500

Nightly Sidekiq job recalculates all scores.

React (6 of 8 components):

- MemberDirectory, MemberProfile, MyTrustScore
- TrustScoreGauge (0-1000 circular gauge), TrustScoreBadge, StatusBadge

Defer: AtRiskMembers, references/dispute workflows

Weeks 8-10: ROSCA Circles (the product)

Budget 3 full weeks — this is the most complex and most critical section.

Week 8: Circle Creation & Management

- Rails: CirclesController, ParticipantsController, Circle state machine (`forming -> active -> completed` via `aasm`), CirclePolicy
- React: MyCirclesDashboard, CircleDetail, CreateCircleWizard (simplify to 4 steps: basics, schedule, payout method, review), InviteMembers

Week 9: Contributions & Payouts

- Rails: ContributionsController, PayoutsController
- `StripeService` : Connected accounts (Standard), payment intents, transfers, webhook handling
- `ContributionReminderJob` : 7d, 3d, 1d before due date
- `PayoutDisbursementJob` : automatic on scheduled date
- `EmergencyFundService` : 1% pooling
- React: ContributionFlow, PayoutSchedule, EmergencyFundPanel

Week 10: Circle Operations

- `CircleCycleService` : advance cycle, calculate next recipient
- Fixed rotation only (defer bidding, lottery, need-based)
- EmergencyFundIntervention auto-cover
- React: TreasurerDashboard, CircleManagement, PositionSwap

Defer: BiddingInterface, CashCollection, MemberRiskScores, DisputeManagement, WaitlistManagement, CircleDiscovery

Week 11: Treasury

Rails: TreasuryController, TransactionsController. All financial movements auto-create Transaction records.

React (4 of 8): TreasuryDashboard, CircleFundDetail, TransactionLedger, EmergencyFundPanel

Defer: ReconciliationConsole, InvestmentManager, MultiCurrencySettings, AuditReportGenerator

Week 12: Notifications + Integration Testing

Rails: NotificationService, Action Cable for real-time in-app notifications, payment reminder jobs

React: Notification bell in shell, notification dropdown, basic preferences

Integration test: Register -> Create Association -> Invite Member -> Create Circle -> Make Contribution -> Receive Payout (end-to-end in Stripe test mode)

Phase 1 Done When:

- User can register, create association, invite members
 - Members create and join ROSCA circles
 - Contributions collected via Stripe, payouts disbursed on schedule (fixed rotation)
 - Emergency Fund auto-covers defaults
 - Trust Scores calculated and displayed
 - Treasury shows all financial movements
 - Notifications remind users of upcoming payments
 - Works on mobile
-

Phase 2: MVP Polish (Months 4-5, Weeks 13-20)

Weeks 13-14: Governance & Voting (simplified)

Only proposals + voting — defer elections entirely.

Rails: ProposalsController, VotesController, Proposal state machine (draft -> voting -> passed/rejected), simple majority only

React (4 of 8): GovernanceDashboard, ProposalCenter, VotingBooth, DecisionArchive

Week 15: Documents (simplified)

Rails: DocumentsController, Active Storage with S3 backend, folder organization

React (3 of 6): DocumentLibrary, DocumentUpload, DocumentViewer

Week 16: Analytics (personal dashboard)

Rails: AnalyticsController, aggregate queries, PDF statement generation (`prawn` or `wicked_pdf`)

React (3 of 7): PersonalDashboard, CircleHealthDashboard, Statements

Weeks 17-18: Stripe Connect Production

- Apply for Stripe Connect production access (apply in Week 1!)
- KYC flow for organizers (Stripe Connect onboarding)
- Production webhooks with signature verification
- Idempotency keys, retry logic, refund handling

Week 19: Email Notifications

Rails: Action Mailer + SendGrid — welcome, contribution reminders, payout notifications, password reset

React: Unified notification center, notification preferences

Defer all messaging: Conversations, ChatView, Announcements — users have WhatsApp. Focus on structured notifications.

Week 20: Mobile & Performance Audit

- Test at 375px, 768px, 1024px, 1440px
 - Lighthouse target: 90+
 - Accessibility audit (keyboard nav, screen readers)
 - Critical bug fixes
-

Phase 3: Launch Prep (Month 6, Weeks 21-24)

Weeks 21-22: Landing Page

React (10 of 15 Homepage components):

- HeroSection, BenefitsSection, HowItWorksSection, StatsSection
- TestimonialsSection, ComparisonSection (ROSCA vs CircleUp)
- PricingSection, FaqSection, Footer, Homepage wrapper

SEO: meta tags, Open Graph, JSON-LD, sitemap

Week 23: Beta Testing

- Recruit **3-5 real diaspora associations** (personal outreach to community leaders)
- Free access for 6 months
- Weekly 30-min feedback calls
- Run a complete circle cycle with real money (small amounts: CHF 10-50)
- In-app feedback widget (Canny.io free tier)

Week 24: Go-Live

Legal: Terms of Service, Privacy Policy, GDPR documentation, cookie consent

Production hardening: `rack-attack` rate limiting, CSP headers, connection pooling, Redis limits, log aggregation

Monitoring: UptimeRobot, Sentry alerting, Plausible analytics (~\$9/month)

Launch: Production deploy, custom domain, SSL, backup verification, rollback plan

Phase 3 Done When: CircleUp live at production URL, 3+ associations active, first real circle collecting contributions, no P0 bugs.

Post-MVP Phases (Months 7-18)

Phase 4: Growth (Months 7-9)

- **Communication & Events** — full build (announcements, events, consider Stream/Ably for messaging)
- **Migration Wizard** — import existing associations from spreadsheets (biggest growth lever)
- **Elections** — complete Governance section
- SMS via Twilio, push via Firebase

Phase 5: Revenue (Months 10-13)

- **Credit & Lending** — payout advances (3-5% fee), credit scores, loan center
- **Multi-Share** — premium feature for power users
- **Projects & Fundraising** — donation campaigns
- Note: Payout advances are the primary revenue driver. Consult Swiss fintech lawyer re: FINMA before full loans.

Phase 6: Intelligence & Scale (Months 14-18)

- **AI Insights** — recommendations, fraud detection, financial health chat (OpenAI API)
- **Community & Social** — referrals (key growth), badges, leaderboards, challenges
- **Federations** — multi-association governance (unlocks enterprise sales)
- **Platform Administration** — build when you hire first support person
- **Analytics full build** — association dashboards, federation analytics, scheduled reports
- Upgrade Trust Score to ML model

Revenue Model

Freemium Tiers

Tier	Price	Includes
Free	CHF 0	1 association, 25 members, 2 circles, fixed rotation, email notifications, CircleUp branding
Community	CHF 29/mo	Unlimited members, 10 circles, all payout methods, 5GB docs, governance, no branding
Professional	CHF 79/mo	Unlimited circles, Credit & Lending, Multi-Share, analytics, API access
Federation	CHF 199/mo	Multi-association, cross-org analytics, compliance reports, dedicated support

Transaction Revenue (Phase 5+)

- Platform fee on contributions: 1.5%
- Payout advance fee: 3-5%
- Loan origination: 1-2%

Pricing Strategy

- Generous free tier to build network effects
- 20% annual discount to reduce churn
- 50% nonprofit discount
- Referral: 1 month free per referred association

Conservative Revenue Projection

Month	Associations	Paid	MRR (CHF)
6	5	0	0
9	20	5	145

12	50	15	535
15	120	40	1,560
18	250	80	3,920

Solo Dev Survival Guide

Automate Early (Week 1)

- CI/CD on every PR
- Automated daily DB backups
- Sentry error alerts to Slack/phone
- Dependabot for dependency updates

Leverage Rails Conventions

- Model generators with migrations
- Controller scaffolding
- Background jobs via `rails g job`
- Mailers via `rails g mailer`

Stub, Don't Build

- Trust Score: weighted formula, not ML
- AI Insights: hardcoded rules
- SMS/WhatsApp: email-only until paying users
- Multi-language: i18n structure from day 1, English-only content
- Offline capability: defer entirely

Use AI Coding Assistants For

1. Rails boilerplate (models, controllers, serializers, specs)
2. Translating Design OS components to production React
3. Test writing (RSpec + React Testing Library)
4. Stripe integration (well-documented patterns)
5. Debugging and SQL optimization

Top Risks

Risk	Mitigation
Stripe Connect approval delayed	Apply Week 1, use test mode until approved
Feature creep from beta users	Rigid MVP scope, say "post-MVP" liberally
Payment bugs with real money	Extensive test mode, start with CHF 10-50 circles
Burnout	40hr weeks, no weekends, clear phase boundaries
Security vulnerability	Established gems (Devise, Stripe), pentest before launch

17 sections feels overwhelming

MVP is only 6 sections. That's the whole product for 6 months.

Key Design OS Reference Files

When implementing each section, reference these files from the Design OS project:

- `product/data-model/data-model.md` — Entity definitions -> Rails models
- `product/design-system/colors.json` + `typography.json` — Design tokens
- `product/shell/spec.md` — Shell layout, RBAC nav, responsive breakpoints
- `product/sections/[section]/spec.md` — Section specification with user flows
- `product/sections/[section]/types.ts` — TypeScript interfaces (reuse in React)
- `product/sections/[section]/data.json` — Sample data for seeding
- `src/sections/[section]/components/` — Reference React implementations (169 components)

The Bottom Line

The absolute minimum viable product is: Auth + Associations + Members + Circles + Treasury with Stripe payments working end-to-end. If you fall behind, cut Governance, Documents, and Analytics from initial launch — add them in month 7.

The one thing you cannot cut: The contribution -> payout loop working reliably with real money. That is the product.

Week 1 action items:

1. Set up Rails + React project with CI/CD
2. Apply for Stripe Connect (takes 1-2 weeks)
3. Deploy staging environment
4. Start building User model and auth